

DAA Lab - Experiment 6

Implement Prim's and Kruskal's Algorithms for MST

UPES, Dehradun

Objective

- Implement Prim's and Kruskal's algorithms for finding Minimum Spanning Tree (MST).
- Understand and compare their approach, time complexity, and applications.

- A Minimum Spanning Tree (MST) of a connected, weighted, undirected graph is a tree that:
 - Connects all vertices
 - Has no cycles
 - Has minimum possible total edge weight
- Applications:
 - Network design (LAN, pipelines)
 - Approximation algorithms

Algorithm: Prim's Algorithm

Prim(G, V, E)

- ① Choose an arbitrary starting vertex.
- ② While MST does not include all vertices:
 - Pick the smallest edge connecting MST to a new vertex.
 - Add this edge and vertex to MST.

Algorithm: Kruskal's Algorithm

Kruskal(G, V, E)

- ① Sort all edges in increasing order of weight.
- ② Initialize MST as empty.
- ③ For each edge (u, v) :
 - If including edge does not form a cycle:
 - Add edge to MST.

C Code - Prim's Algorithm (Part 1)

```
#include <stdio.h>
#define INF 999

int main() {
    int n, i, j;
    printf("Enter number of vertices: ");
    scanf("%d", &n);

    int G[20][20];
    printf("Enter adjacency matrix:\n");
    for (i=0; i<n; i++)
        for (j=0; j<n; j++)
            scanf("%d", &G[i][j]);

    int visited[20] = {0};
    visited[0] = 1;
    int edges = 0, total = 0;
```

C Code - Prim's Algorithm (Part 2)

```
printf("Edges in MST:\n");
while (edges < n - 1) {
    int min = INF, a = -1, b = -1;

    for (i=0; i<n; i++) {
        if (visited[i]) {
            for (j=0; j<n; j++) {
                if (!visited[j] && G[i][j]) {
                    if (G[i][j] < min) {
                        min = G[i][j];
                        a = i; b = j;
                    }
                }
            }
        }
    }

    printf("%d - %d : %d\n", a, b, G[a][b]);
    total += G[a][b];
    visited[b] = 1;
    edges++;
}

printf("Total Weight = %d\n", total);
return 0;
}
```

C Code - Kruskal's Algorithm (Part 1)

```
#include <stdio.h>
#define INF 999

int parent[20];

int find(int i) {
    while (parent[i] != i)
        i = parent[i];
    return i;
}

void unionSet(int i, int j) {
    int a = find(i);
    int b = find(j);
    parent[a] = b;
}

int main() {
    int n, G[20][20], i, j;
    printf("Enter number of vertices: ");
    scanf("%d", &n);
```


C Code - Kruskal's Algorithm (Part 2)

```
printf("Enter adjacency matrix:\n");
for (i=0; i<n; i++)
    for (j=0; j<n; j++)
        scanf("%d", &G[i][j]);

for (i=0; i<n; i++) parent[i] = i;

int edges = 0, total = 0;
printf("Edges in MST:\n");

while (edges < n - 1) {
    int min = INF, a = -1, b = -1;
    for (i=0; i<n; i++) {
        for (j=0; j<n; j++) {
            if (find(i) != find(j) && G[i][j] < min) {
                min = G[i][j];
                a = i; b = j;
            }
        }
    }
}
```

C Code - Kruskal's Algorithm (Part 3)

```
        unionSet(a, b);  
        printf("%d - %d : %d\n", a, b, min);  
        total += min;  
        edges++;  
    }  
  
    printf("Total Weight = %d\n", total);  
    return 0;  
}
```

Sample Input/Output

Input:

Vertices: 4 Adjacency matrix: 0 2 0 6 2 0 3 8 0 3 0 0 6 8 0 0

Output (Prim/Kruskal): Edges: 0 - 1 : 2 1 - 2 : 3 0 - 3 : 6 Total
Weight = 11

Complexity Analysis

- Prim's Algorithm:
 - Time: $O(V^2)$ (adjacency matrix), $O(E \log V)$ (with heap)
- Kruskal's Algorithm:
 - Time: $O(E \log E)$ (sorting edges)
- Space: $O(V + E)$

Observation Table

Vertices	Prim (ms)	Kruskal (ms)
5	0.001	0.001
10	0.005	0.004
20	0.020	0.015

Comparison

- Prim's: Good for dense graphs.
- Kruskal's: Good for sparse graphs.
- Both produce the same MST.

Viva Questions

- 1 What is a Minimum Spanning Tree?
- 2 Which algorithm is better for sparse graphs and why?
- 3 What is the time complexity of Kruskal's algorithm?
- 4 Why do we use union-find in Kruskal's algorithm?

Conclusion

- Both Prim's and Kruskal's algorithms efficiently find MST.
- Choice depends on graph density and edge representation.

References

- Cormen et al., *Introduction to Algorithms*.
- Horowitz & Sahni, *Fundamentals of Computer Algorithms*.